Sesión 4

Control de Accesos y Sistema Anti-Trampas

Esta sesión trabaja tres protecciones sobre la base `juego_online`: evitar datos absurdos, limitar lo que cada perfil puede hacer y frenar la inyección SQL.

1. Restricciones CHECK: candados para datos coherentes

Para qué sirven: impiden que entren valores imposibles al motor, aunque la aplicación tenga un bug o alguien intente manipularla.

Idea clave: si un jugador no puede tener monedas negativas, nivel 0 o un resultado inventado, la propia base de datos lo rechaza.

SQL esencial:

```
sql
USE juego_online;

ALTER TABLE Jugadores
  ADD CONSTRAINT chk_monedas_no_negativas CHECK (monedas >= 0),
  ADD CONSTRAINT chk_experiencia_no_negativa CHECK (experiencia >= 0),
  ADD CONSTRAINT chk_nivel_minimo CHECK (nivel >= 1);

ALTER TABLE Partidas
  ADD CONSTRAINT chk_puntaje_no_negativo CHECK (puntaje >= 0),
  ADD CONSTRAINT chk_resultado_valido CHECK (resultado IN ('victoria', 'derro
ta', 'jugada'));
```

Resultado esperado: un `UPDATE` como `monedas = -100` falla con error. La fila no se modifica.

Requiere MySQL 8.0.16+ o MariaDB 10.2.1+, porque las versiones antiguas aceptaban `CHECK` pero no lo aplicaban.

2. Roles y permisos: cada perfil hace solo lo suyo

Para qué sirven: aplican el principio de privilegio mínimo. Un jugador no debería poder darse monedas; un moderador no debería asignar dinero; solo el administrador controla todo.

Comparación rápida:

Perfil	Puede	No puede
`rol_jugador`	Consultar catálogos, registrar partidas, ejecutar rutinas de nivel y compra	Modificar monedas o `UPDATE` directo
`rol_moderador`	Hereda lo del jugador y puede banear (`UPDATE (baneado)`)	Asignar monedas
`rol_administrador`	Hereda lo del moderador y controla monedas, nivel, experiencia y borrados	—

SQL esencial:

```
sql
ALTER TABLE Jugadores ADD COLUMN IF NOT EXISTS baneado BOOLEAN NOT NULL DEFAU
LT FALSE;

DROP ROLE IF EXISTS rol_jugador;
CREATE ROLE rol_jugador;
GRANT SELECT ON juego_online.Jugadores TO rol_jugador;
GRANT EXECUTE ON PROCEDURE juego_online.subir_nivel_jugador TO rol_jugador;
GRANT EXECUTE ON PROCEDURE juego_online.comprar_item_unico TO rol_jugador;

DROP ROLE IF EXISTS rol_moderador;
CREATE ROLE rol_moderador;
GRANT rol_jugador TO rol_moderador;
GRANT UPDATE (baneado) ON juego_online.Jugadores TO rol_moderador;

DROP ROLE IF EXISTS rol_administrador;
CREATE ROLE rol_administrador;
GRANT rol_moderador TO rol_administrador;
GRANT UPDATE (monedas, nivel, experiencia) ON juego_online.Jugadores TO rol_a
dministrador;

FLUSH PRIVILEGES;
```

Resultado esperado: `SHOW GRANTS FOR rol_administrador;` muestra los permisos propios y los heredados.

Necesita que existan las rutinas `subir_nivel_jugador` y `comprar_item_unico` de sesiones anteriores.

3. Anti-inyección SQL: el texto del usuario nunca es código

Para qué sirve: evita que alguien escriba SQL malicioso en un buscador o login y logre leer, borrar o modificar datos.

Problema: si el backend pega el texto del usuario dentro de la consulta, un valor como `' OR '1'='1` puede devolver toda la tabla.

Solución: usar un procedimiento con parámetro tipado, o consultas preparadas en el backend.

SQL esencial:

```
sql
DROP PROCEDURE IF EXISTS buscar_jugador_seguro;

DELIMITER //
CREATE PROCEDURE buscar_jugador_seguro(IN p_nombre_usuario VARCHAR(50))
BEGIN
  SELECT jugador_id, nombre_usuario, nivel, monedas
  FROM Jugadores
  WHERE nombre_usuario = p_nombre_usuario;
END //
DELIMITER
```

Pruebas:

```
sql
CALL buscar_jugador_seguro('\' OR \'1\'=\'1');

CALL buscar_jugador_seguro('DragonSlayer99');
```

Conclusión: en el backend usa siempre consultas preparadas o parámetros vinculados. Nunca armes SQL concatenando texto del usuario

Cierre

- **CHECK** protege el significado de los datos.
- **Roles** limitan lo que cada cuenta puede hacer.
- **Procedimientos y parámetros** bloquean la inyección SQL.